

AI Research Paper Intelligence System

Data collection

text extraction

NLP preprocessing and cleaning

embedding

transformer

Facebook AI Similarity Search(Faiss) vector database

Semantic search

Summarization using Bi directional and Auto-Regressive Model(BART)

Implementing Key BERT

```
!pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.29.4)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.7.1)
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (2025.3.0)
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.36.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!<4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (4.0.0a1)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (1.1.3)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (4.12.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->dataset) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->dataset) (2025.11.11)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2026.2)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1->fsspec) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (6.1.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (0.3.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (1.20.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->dataset) (1.17.0)
```

```
from datasets import load_dataset
```

```
dataset=load_dataset("CShorten/ML-Arxiv-Papers",split='train')
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as an environment variable `HF_TOKEN` or store it as a Colab secret.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
```

```
print(dataset)
```

```
Dataset({
  features: ['Unnamed: 0.1', 'Unnamed: 0', 'title', 'abstract'],
  num_rows: 117592
})
```

```
dataset[0]
```

```
{'Unnamed: 0.1': 0,
 'Unnamed: 0': 0.0,
 'title': 'Learning from compressed observations',
 'abstract': 'The problem of statistical learning is to construct a predictor of a random variable $Y$ as a function of a related random variable $X$ on the basis of an $n$ i.i.d. training sample from the joint distribution of $(X,Y)$. Allowable predictors are drawn from some specified class, and the goal is to approach asymptotically the performance
```

(expected loss) of the best predictor in the class. We consider the setting in which one has perfect observation of the X -part of the sample, while the Y -part has to be communicated at some infinite bit rate. The encoding of the Y -values is allowed to depend on the X -values. Under suitable regularity conditions on the admissible predictors, the underlying family of probability distributions and the loss function, we give an information-theoretic characterization of achievable predictor performance in terms of conditional distortion-rate functions. The ideas are illustrated on the example of nonparametric regression in Gaussian noise.

```
import pandas as pd
```

Made Dataframe

```
df=pd.DataFrame(dataset)
df
```

	Unnamed: 0.1	Unnamed: 0	title	abstract
0	0	0.0	Learning from compressed observations	The problem of statistical learning is to co...
1	1	1.0	Sensor Networks with Random Links: Topology De...	In a sensor network, in practice, the commun...
2	2	2.0	The on-line shortest path problem under partia...	The on-line shortest path problem is conside...
3	3	3.0	A neural network approach to ordinal regression	Ordinal regression is an important type of l...
4	4	4.0	Parametric Learning and Monte Carlo Optimization	This paper uncovers and explores the close r...
...	...	...	...	...
117587	4995	NaN	Detecting COVID-19 Conspiracy Theories with Tr...	The sharing of fake news and conspiracy theori...
117588	4996	NaN	Fair Feature Subset Selection using Multiobjec...	The feature subset selection problem aims at s...
117589	4997	NaN	A Simple Duality Proof for Wasserstein Distrib...	We present a short and elementary proof of the...
117590	4998	NaN	Combined Learning of Neural Network Weights fo...	We introduce CoLN, Combined Learning of

```
df.columns
```

```
Index(['Unnamed: 0.1', 'Unnamed: 0', 'title', 'abstract'], dtype='object')
```

Drop title and abstract

```
df=df[['title', 'abstract']]
```

```
df
```

	title	abstract
0	Learning from compressed observations	The problem of statistical learning is to co...
1	Sensor Networks with Random Links: Topology De...	In a sensor network, in practice, the commun...
2	The on-line shortest path problem under partia...	The on-line shortest path problem is conside...
3	A neural network approach to ordinal regression	Ordinal regression is an important type of l...
4	Parametric Learning and Monte Carlo Optimization	This paper uncovers and explores the close r...
...	...	...
117587	Detecting COVID-19 Conspiracy Theories with Tr...	The sharing of fake news and conspiracy theori...
117588	Fair Feature Subset Selection using Multiobjec...	The feature subset selection problem aims at s...
117589	A Simple Duality Proof for Wasserstein Distrib...	We present a short and elementary proof of the...
117590	Combined Learning of Neural Network Weights fo...	We introduce CoLN, Combined Learning of Neural...
117591	Adapting and Evaluating Influence-Estimation M...	Influence estimation analyzes how changes to t...

117592 rows × 2 columns

```
df.shape
```

```
(117592, 2)
```

Decrease data

```
df=df.head(15000)
```

```
df.shape
```

```
(15000, 2)
```

```
df.isnull().sum()
```

```

      0
title  0
abstract 0
dtype: int64

```

new column

```
df["paper_text"]=df["title"]+" "+df["abstract"]
```

```

/tmp/ipykernel_3840/529180721.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view

```
df["paper_text"]=df["title"]+" "+df["abstract"]
```

```
df[["paper_text"]].head()
```

	paper_text 
0	Learning from compressed observations The pr...
1	Sensor Networks with Random Links: Topology De...
2	The on-line shortest path problem under partia...
3	A neural network approach to ordinal regressio...
4	Parametric Learning and Monte Carlo Optimizati...

```
type(df[["paper_text"]])
```

```

pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype |
None=None, copy: bool | None=None) -> None

```

</usr/local/lib/python3.12/dist-packages/pandas/core/frame.py>
Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns).
Arithmetic operations align on both row and column labels. Can be
thought of as a dict-like container for Series objects. The primary

```
print(df["paper_text"].iloc[0])
```

Learning from compressed observations The problem of statistical learning is to construct a predictor of a random variable Y as a function of a related random variable X on the basis of an i.i.d. training sample from the joint distribution of (X, Y) . Allowable predictors are drawn from some specified class, and the goal is to approach asymptotically the performance (expected loss) of the best predictor in the class. We consider the setting in which one has perfect observation of the X -part of the sample, while the Y -part has to be communicated at some finite bit rate. The encoding of the Y -values is allowed to depend on the X -values. Under suitable regularity conditions on the admissible predictors, the underlying family of probability distributions and the loss function, we give an information-theoretic characterization of achievable predictor performance in terms of conditional distortion-rate functions. The ideas are illustrated on the example of nonparametric regression in Gaussian noise.

```
from sentence_transformers import SentenceTransformer
```

The cache for model files in Transformers v4.22.0 has been updated. Migrating your old cache. This is a one-time only operation
0/0 [00:00<?, ?it/s]

WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels will not work

used transformer to extract info

```
model=SentenceTransformer("all-MiniLM-L6-v2")
```

```
print(type(model))
```

```
<class 'sentence_transformers.sentence_transformer.model.SentenceTransformer'>
```

```
sample_text=df["paper_text"].iloc[0]
```

```
sample_text
```

```
'Learning from compressed observations The problem of statistical learning is to construct a predictor of a random variable $Y$ as a function of a related random variable $X$ on the basis of an i.i.d. training sample from the joint distribution of $(X,Y)$. Allowable predictors are drawn from some specified class, and the goal is to approach asymptotically the performance (expected loss) of the best predictor in the class. We consider the setting in which one has perfect observation of the $X$-part of the sample, while the $Y$-part has to be communicated at some finite bit rate. The encoding of the $Y$-values is allowed to depend on the $X$-values. Under suitable regularity conditions on the admissible predictors, the underlying family of probability distributions and the loss function, we give an information-theoretic characterization of achievable predictor performance in terms of conditional distortion-rate functions. The ideas are illustrated on the exam
```

replaced \n and space

```
df["paper_text"]=df["paper_text"].str.replace("\n"," ",regex=False)
df["paper_text"]=df["paper_text"].str.strip()
```

```
/tmp/ipykernel_3840/2190359946.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view
df["paper_text"]=df["paper_text"].str.replace("\n"," ",regex=False)
```

```
/tmp/ipykernel_3840/2190359946.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view
df["paper_text"]=df["paper_text"].str.strip()
```

```
sample_text=df["paper_text"].iloc[0]
sample_text
```

```
'Learning from compressed observations The problem of statistical learning is to construct a predictor of a random variable $Y$ as a function of a related random variable $X$ on the basis of an i.i.d. training sample from the joint distribution of $(X,Y)$. Allowable predictors are drawn from some specified class, and the goal is to approach asymptotically the performance (expected loss) of the best predictor in the class. We consider the setting in which one has perfect observation of the $X$-part of the sample, while the $Y$-part has to be communicated at some finite bit rate. The encoding of the $Y$-values is allowed to depend on the $X$-values. Under suitable regularity conditions on the admissible predictors, the underlying family of probability distributions and the loss function, we give an information-theoretic characterization of achievable predictor performance in terms of conditional distortion-rate functions. The ideas are illustrated on the example of nonpar
```

```
embedding=model.encode(sample_text)
print(type(embedding))
print(embedding.shape)
```

```
<class 'numpy.ndarray'>
(384,)
```

```
embedding[:56]
```

```
array([-0.1315641 , -0.00678266, -0.00367612,  0.03265158,  0.11219642,
        0.01227267,  0.09816719, -0.0900523 ,  0.04231161, -0.01977348,
       -0.03308417,  0.07452948,  0.10632038, -0.02060429, -0.02052106,
        0.00169493,  0.07081953,  0.05854454, -0.11231912,  0.02082474,
        0.05692544,  0.0201578 ,  0.0258311 ,  0.0321703 ,  0.10513764,
       -0.09676763,  0.02700802, -0.0234509 , -0.04549678, -0.01013699,
       -0.01794855, -0.04814427,  0.01077652, -0.03759069,  0.01943481,
        0.03715189,  0.02967844,  0.04330941,  0.04373213,  0.03704866,
       -0.00182594,  0.00455183, -0.00799067,  0.03037368, -0.014378 ,
        0.03795147,  0.0595916 , -0.02583356, -0.06521576,  0.05900268,
       -0.02107134,  0.07359422, -0.05720106,  0.00294061,  0.00767515,
       -0.0333416 ], dtype=float32)
```

```
sample_embedding=model.encode(df["paper_text"].head(5).to_list())
```

```
print(sample_embedding.shape)
```

```
(5, 384)
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
similarity=cosine_similarity(sample_embedding[0].reshape(1,-1),sample_embedding[0].reshape(1,-1))
print(similarity)
```

```
[[1.0000001]]
```

```
similarity=cosine_similarity(sample_embedding[0].reshape(1,-1),sample_embedding[1].reshape(1,-1))
print(similarity)
```

```
[[0.36625272]]
```

```
for i in range(1,5):
    sim=cosine_similarity(sample_embedding[0].reshape(1,-1),sample_embedding[i].reshape(1,-1))
    print(sim)
```

```
[[0.36625272]]
[[0.33522844]]
[[0.15505108]]
[[0.37421533]]
```

Generate Full Embedding of all Papers

```
import numpy as np
import os
if(os.path.exists("embedding.npy")):
    print("File Already Exists")
    embedding=np.load("embedding.npy")
else:
    print("Generating Embedding")
    embedding=model.encode(df["paper_text"].to_list(),batch_size=32,show_progress_bar=True)
```

```
File Already Exists
```

```
import numpy as np
np.save("embedding.npy",embedding)
```

```
print(embedding.shape)
print(type(embedding))
```

```
(15000, 384)
<class 'numpy.ndarray'>
```

```
!pip install faiss-cpu
```

```
Requirement already satisfied: faiss-cpu in /usr/local/lib/python3.12/dist-packages (1.14.3)
Requirement already satisfied: numpy>=1.25 in /usr/local/lib/python3.12/dist-packages (from faiss-cpu) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from faiss-cpu) (26.2)
```

```
import faiss
```

```
faiss.normalize_L2(embedding)
```

```
index=faiss.IndexFlatIP(384)
```

```
index.add(embedding)
```

```
print(index.ntotal)
```

```
15000
```

```
query="deep learning for medical image analysis"
query_embedding=model.encode([query])
query_embedding.shape
```

```
(1, 384)
```

```
faiss.normalize_L2(query_embedding)
```

```
D,I=index.search(query_embedding,5)
print(D) #Score / Similarity score
print(I) #Indices
```

```
[[0.6807244 0.67092204 0.65219975 0.62811744 0.61311525]]
[[10466 13730 11873 12691 11282]]
```

```
print(df.iloc[10466]["title"])
```

A Perspective on Deep Imaging

```
print(df.iloc[10466]["abstract"])
```

The combination of tomographic imaging and deep learning, or machine learning in general, promises to empower not only image analysis but also image reconstruction. The latter aspect is considered in this perspective article with an emphasis on medical imaging to develop a new generation of image reconstruction theories and techniques. This direction might lead to intelligent utilization of domain knowledge from big data, innovative approaches for image reconstruction, and superior performance in clinical and preclinical applications. To realize the full impact of machine learning on medical imaging, major challenges must be addressed.

```
print(df.iloc[11873]["title"])
```

Classification of MRI data using Deep Learning and Gaussian Process-based Model Selection

```
def search_paper(query,k=5):
    query_embedding=model.encode([query])
    faiss.normalize_L2(query_embedding)
    D,I=index.search(query_embedding,k)
    return D,I
```

```
D,I=search_paper("deep learning for medical image analysis")
print(D)
print(I)
```

```
[[0.6807244 0.67092204 0.65219975 0.62811744 0.61311525]]
[[10466 13730 11873 12691 11282]]
```

```
def search_paper(query,k=5):
    query_embedding=model.encode([query])
    faiss.normalize_L2(query_embedding)
    D,I=index.search(query_embedding,k)
    for score,idx in zip(D[0],I[0]):
        print("Similarity score",score)
        print("Title",df.iloc[idx]["title"])
        print("Abstract",df.iloc[idx]["abstract"][:500])
        print()
```

```
search_paper("deep learning for medical image analysis")
```

Similarity score 0.6807244
 Title A Perspective on Deep Imaging
 Abstract The combination of tomographic imaging and deep learning, or machine learning in general, promises to empower not only image analysis but also image reconstruction. The latter aspect is considered in this perspective article with an emphasis on medical imaging to develop a new generation of image reconstruction theories and techniques. This direction might lead to intelligent utilization of domain knowledge from big data, innovative approaches for image reconstruction, and superior performance

Similarity score 0.67092204
 Title Convolutional Neural Networks for Medical Image Analysis: Full Training or Fine Tuning?
 Abstract Training a deep convolutional neural network (CNN) from scratch is difficult because it requires a large amount of labeled training data and a great deal of expertise to ensure proper convergence. A promising alternative is to fine-tune a CNN that has been pre-trained using, for instance, a large set of labeled natural images. However, the substantial differences between natural and medical images may advise against such knowledge transfer. In this paper, we seek to answer the following centra

Similarity score 0.65219975
 Title Classification of MRI data using Deep Learning and Gaussian Process-based Model Selection
 Abstract The classification of MRI images according to the anatomical field of view is a necessary task to solve when faced with the increasing quantity of medical images. In parallel, advances in deep learning makes it a suitable tool for computer vision problems. Using a common architecture (such as AlexNet) provides quite good results, but not sufficient for clinical use. Improving the model is not an easy task, due to the large number of hyper-parameters governing both the architecture and the trai

Similarity score 0.62811744
 Title High-Resolution Breast Cancer Screening with Multi-View Deep Convolutional Neural Networks
 Abstract Advances in deep learning for natural images have prompted a surge of

interest in applying similar techniques to medical images. The majority of the initial attempts focused on replacing the input of a deep convolutional neural network with a medical image, which does not take into consideration the fundamental differences between these two types of images. Specifically, fine details are necessary for detection in medical images, unlike in natural images where coarse structures matter most. Th

Similarity score 0.61311525

Title Deep Learning for the Classification of Lung Nodules

Abstract Deep learning, as a promising new area of machine learning, has attracted a rapidly increasing attention in the field of medical imaging. Compared to the conventional machine learning methods, deep learning requires no hand-tuned feature extractor, and has shown a superior performance in many visual object recognition applications. In this study, we develop a deep convolutional neural network (CNN) and apply it to thoracic CT images for the classification of lung nodules. We present the CNN ar

Summarization

```
!pip install transformers==4.46.3
from transformers import pipeline
summarizer=pipeline(
    "summarization",
    model="facebook/bart-large-cnn",
)
```

Requirement already satisfied: transformers==4.46.3 in /usr/local/lib/python3.12/dist-packages (4.46.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (3.29.4)
Requirement already satisfied: huggingface-hub<1.0,>=0.23.2 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (0.23.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2022.10.31)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.32.4)
Requirement already satisfied: tokenizers<0.21,>=0.20 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (0.20.3)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (0.4.3)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (4.67.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.23.2) (2023.12.2)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.23.2) (1.1.3)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.23.2) (4.11.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (2024.7.4)

config.json: 1.58k/? [00:00<00:00, 110kB/s]

model.safetensors: 100% 1.63G/1.63G [00:16<00:00, 91.6MB/s]

generation_config.json: 100% 363/363 [00:00<00:00, 30.2kB/s]

vocab.json: 899k/? [00:00<00:00, 3.12MB/s]

merges.txt: 456k/? [00:00<00:00, 2.80MB/s]

tokenizer.json: 1.36M/? [00:00<00:00, 2.87MB/s]

```
type(summarizer)
```

```
transformers.pipelines.text2text_generation.SummarizationPipeline
def __call__(*args, **kwargs)
```

/usr/local/lib/python3.12/dist-packages/transformers/pipelines/text2text_generation.py
Summarize news articles and other documents.

This summarizing pipeline can currently be loaded from [`pipeline`] using the following task identifier "summarization".

```
summary=summarizer(df.iloc[10466]["abstract"],max_length=120,min_length=40)
print(summary)
```

```
[{'summary_text': 'The combination of tomographic imaging and deep learning, or machine learning, promises to empower not or
```

```
type(summary)
```

```
list
```

```
type(summary[0])
```

```
dict
```

```
summary[0]["summary_text"]
```

'The combination of tomographic imaging and deep learning, or machine learning, promises to empower not only image analysis but also image reconstructions. This direction might lead to intelligent utilization of domain knowledge from big data, innovative approaches for image reconstruction, and superior performance in clinical applications.'

```
for score,idx in zip(D[0],I[0]):
    print("Similarity score",score)
    print("Title",df.iloc[idx]["title"])
    print("Abstract",df.iloc[idx]["abstract"][:500])
    print()
    summary=summarizer(df.iloc[idx]["abstract"],max_length=120,min_length=40)
    print(summary)
    print(summary[0]["summary_text"])
    print()
```

```
[{'summary_text': 'The combination of tomographic imaging and deep learning, or machine learning, promises to empower not only image analysis but also image reconstructions. This direction might lead to intelligent utilization of domain knowledge from big data, innovative approaches for image reconstruction, and superior performance in clinical applications.'}]
```

```
Similarity score 0.67092204
Title Convolutional Neural Networks for Medical Image Analysis: Full Training or Fine Tuning?
```

Abstract Training a deep convolutional neural network (CNN) from scratch is difficult because it requires a large amount of labeled training data and a great deal of expertise to ensure proper convergence. A promising alternative is to fine-tune a CNN that has been pre-trained using, for instance, a large set of labeled natural images. However, the substantial differences between natural and medical images may advise against such knowledge transfer. In this paper, we seek to answer the following central question: Can knowledge from natural images be transferred to medical images for image classification?

```
[{'summary_text': 'Training a deep convolutional neural network from scratch is difficult because it requires a large amount of labeled training data and a great deal of expertise to ensure proper convergence. A promising alternative is to fine-tune a CNN that has been pre-trained using, for instance, a large set of labeled natural images. However, the substantial differences between natural and medical images may advise against such knowledge transfer. In this paper, we seek to answer the following central question: Can knowledge from natural images be transferred to medical images for image classification?'}]
```

```
Similarity score 0.65219975
Title Classification of MRI data using Deep Learning and Gaussian Process-based Model Selection
```

Abstract The classification of MRI images according to the anatomical field of view is a necessary task to solve when faced with the increasing quantity of medical images. In parallel, advances in deep learning makes it a suitable tool for computer vision problems. Using a common architecture (such as AlexNet) provides quite good results, but not sufficient for clinical use. Improving the model is not an easy task, due to the large number of hyper-parameters governing both the architecture and the training process.

```
[{'summary_text': 'The classification of MRI images according to the anatomical field of view is a necessary task to solve when faced with the increasing quantity of medical images. In parallel, advances in deep learning makes it a suitable tool for computer vision problems. Using a common architecture (such as AlexNet) provides quite good results, but not sufficient for clinical use. Improving the model is not an easy task, due to the large number of hyper-parameters governing both the architecture and the training process.'}]
```

```
Similarity score 0.62811744
Title High-Resolution Breast Cancer Screening with Multi-View Deep Convolutional Neural Networks
```

Abstract Advances in deep learning for natural images have prompted a surge of interest in applying similar techniques to medical images. The majority of the initial attempts focused on replacing the input of a deep convolutional neural network with a medical image, which does not take into consideration the fundamental differences between these two types of images. Specifically, fine details are necessary for detection in medical images, unlike in natural images where coarse structures matter most. This paper presents a multi-view deep convolutional neural network for high-resolution breast cancer screening.

```
[{'summary_text': 'The study used a set of high-resolution images to test the accuracy of the neural network. It found that the network performed well in detecting breast cancer lesions, even when the input images were of low resolution. This suggests that the multi-view deep convolutional neural network is a promising tool for high-resolution breast cancer screening.'}]
```

```
Similarity score 0.61311525
Title Deep Learning for the Classification of Lung Nodules
```

Abstract Deep learning, as a promising new area of machine learning, has attracted a rapidly increasing attention in the field of medical imaging. Compared to the conventional machine learning methods, deep learning requires no hand-tuned feature extractor, and has shown a superior performance in many visual object recognition applications. In this study, we develop a deep convolutional neural network (CNN) and apply it to thoracic CT images for the classification of lung nodules. We present the CNN architecture and the experimental results.

```
[{'summary_text': 'Deep learning, as a promising new area of machine learning, has attracted a rapidly increasing attention in the field of medical imaging. Compared to the conventional machine learning methods, deep learning requires no hand-tuned feature extractor, and has shown a superior performance in many visual object recognition applications. In this study, we develop a deep convolutional neural network (CNN) and apply it to thoracic CT images for the classification of lung nodules. We present the CNN architecture and the experimental results.'}]
```

```
def search_and_summarize(query,k=5):
    query_embedding=model.encode([query])
    faiss.normalize_L2(query_embedding)
    D,I=index.search(query_embedding,k)
    for score,idx in zip(D[0],I[0]):
        print("Similarity score",score)
        print("Title",df.iloc[idx]["title"])
        print("Abstract",df.iloc[idx]["abstract"][:500])
        print()
        summary=summarizer(df.iloc[idx]["abstract"],max_length=120,min_length=40)
        print(summary)
```

```
print(summary[0]['summary_text'])
print()
```

```
search_and_summarize("Deep learning in medical imaging",5)
```

Similarity score 0.73549855

Title A Perspective on Deep Imaging

Abstract The combination of tomographic imaging and deep learning, or machine learning in general, promises to empower not only image analysis but also image reconstruction. The latter aspect is considered in this perspective article with an emphasis on medical imaging to develop a new generation of image reconstruction theories and techniques. This direction might lead to intelligent utilization of domain knowledge from big data, innovative approaches for image reconstruction, and superior performance

```
[{'summary_text': 'The combination of tomographic imaging and deep learning, or machine learning, promises to empower not only image analysis but also image reconstruction. The latter aspect is considered in this perspective article with an emphasis on medical imaging to develop a new generation of image reconstruction theories and techniques. This direction might lead to intelligent utilization of domain knowledge from big data, innovative approaches for image reconstruction, and superior performance'}
```

Similarity score 0.6881736

Title Classification of MRI data using Deep Learning and Gaussian Process-based Model Selection

Abstract The classification of MRI images according to the anatomical field of view is a necessary task to solve when faced with the increasing quantity of medical images. In parallel, advances in deep learning makes it a suitable tool for computer vision problems. Using a common architecture (such as AlexNet) provides quite good results, but not sufficient for clinical use. Improving the model is not an easy task, due to the large number of hyper-parameters governing both the architecture and the training process

```
[{'summary_text': 'The classification of MRI images according to the anatomical field of view is a necessary task to solve when faced with the increasing quantity of medical images. In parallel, advances in deep learning makes it a suitable tool for computer vision problems. Using a common architecture (such as AlexNet) provides quite good results, but not sufficient for clinical use. Improving the model is not an easy task, due to the large number of hyper-parameters governing both the architecture and the training process'}
```

Similarity score 0.6532082

Title Convolutional Neural Networks for Medical Image Analysis: Full Training or Fine Tuning?

Abstract Training a deep convolutional neural network (CNN) from scratch is difficult because it requires a large amount of labeled training data and a great deal of expertise to ensure proper convergence. A promising alternative is to fine-tune a CNN that has been pre-trained using, for instance, a large set of labeled natural images. However, the substantial differences between natural and medical images may advise against such knowledge transfer. In this paper, we seek to answer the following central question: Can we transfer knowledge from natural to medical images?

```
[{'summary_text': 'Training a deep convolutional neural network from scratch is difficult because it requires a large amount of labeled training data and a great deal of expertise to ensure proper convergence. A promising alternative is to fine-tune a CNN that has been pre-trained using, for instance, a large set of labeled natural images. However, the substantial differences between natural and medical images may advise against such knowledge transfer. In this paper, we seek to answer the following central question: Can we transfer knowledge from natural to medical images?'}
```

Similarity score 0.62220883

Title High-Resolution Breast Cancer Screening with Multi-View Deep Convolutional Neural Networks

Abstract Advances in deep learning for natural images have prompted a surge of interest in applying similar techniques to medical images. The majority of the initial attempts focused on replacing the input of a deep convolutional neural network with a medical image, which does not take into consideration the fundamental differences between these two types of images. Specifically, fine details are necessary for detection in medical images, unlike in natural images where coarse structures matter most. This paper proposes a multi-view deep convolutional neural network for high-resolution breast cancer screening. The network is trained on a large set of high-resolution breast cancer screening images. The network is able to detect small lesions in medical images, which is a challenging task for traditional methods.

```
[{'summary_text': 'Advances in deep learning for natural images have prompted a surge of interest in applying similar techniques to medical images. The majority of the initial attempts focused on replacing the input of a deep convolutional neural network with a medical image, which does not take into consideration the fundamental differences between these two types of images. Specifically, fine details are necessary for detection in medical images, unlike in natural images where coarse structures matter most. This paper proposes a multi-view deep convolutional neural network for high-resolution breast cancer screening. The network is trained on a large set of high-resolution breast cancer screening images. The network is able to detect small lesions in medical images, which is a challenging task for traditional methods.'}
```

Similarity score 0.6151297

Title Deep Learning Microscopy

KEY BERT implementation

```
!pip install keybert==0.8
```

Collecting keybert==0.8

Downloading keybert-0.8.0.tar.gz (29 kB)

Preparing metadata (setup.py) ... done

```
Requirement already satisfied: sentence-transformers>=0.3.8 in /usr/local/lib/python3.12/dist-packages (from keybert==0.8) (0.3.8)
Requirement already satisfied: scikit-learn>=0.22.2 in /usr/local/lib/python3.12/dist-packages (from keybert==0.8) (1.6.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from keybert==0.8) (2.0.2)
Requirement already satisfied: rich>=10.4.0 in /usr/local/lib/python3.12/dist-packages (from keybert==0.8) (13.9.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=10.4.0->keybert==0.8) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=10.4.0->keybert==0.8) (2.18.0)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.2->keybert==0.8) (1.12.0)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.2->keybert==0.8) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.2->keybert==0.8) (3.5.0)
Requirement already satisfied: transformers<6.0.0,>=4.41.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers>=0.3.8->keybert==0.8) (4.41.0)
Requirement already satisfied: huggingface-hub>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers>=0.3.8->keybert==0.8) (0.23.0)
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers>=0.3.8->keybert==0.8) (2.4.0)
Requirement already satisfied: typing_extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers>=0.3.8->keybert==0.8) (4.6.3)
Requirement already satisfied: tqdm>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers>=0.3.8->keybert==0.8) (4.66.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers>=0.3.8->keybert==0.8) (3.16.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers>=0.3.8->keybert==0.8) (2024.10.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-transformers>=0.3.8->keybert==0.8) (1.1.3)
```

```
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->ser
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentenc
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.23.0->sentence-t
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=10.4
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transf
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transf
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-trar
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers>
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.
Requirement already satisfied: tokenizers<0.21,>=0.20 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.1
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=1.11.0->sente
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->huggingfa
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface-hub>=0.23
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface-hub
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface-hub
Building wheels for collected packages: keybert
  Building wheel for keybert (setup.py) ... done
  Created wheel for keybert: filename=keybert-0.8.0-py3-none-any.whl size=39142 sha256=42cd3885b9be085a6a9c938d970b1e3f78ce4
  Stored in directory: /root/.cache/pip/wheels/8b/2c/b0/6a4586e1c5221cd77b28a8d38d741425bc69082234ff3bcc30
Successfully built keybert
Installing collected packages: keybert
Successfully installed keybert-0.8.0
```

```
from keybert import KeyBERT
```

```
kw_model=KeyBERT()
```

```
type(kw_model) #SentenceTransformer
```

```
keybert._model.KeyBERT
def __init__(model='all-MiniLM-L6-v2', llm: BaseLLM=None)
```

/usr/local/lib/python3.12/dist-packages/keybert/_model.py
A minimal method for keyword extraction with BERT

The keyword extraction is done by finding the sub-phrases in
a document that are the most similar to the document itself.

```
text=df.iloc[10466]["abstract"]
keywords=kw_model.extract_keywords(text)
```

```
print(keywords)
```

```
[('imaging', 0.4528), ('tomographic', 0.4488), ('reconstruction', 0.3623), ('deep', 0.3003), ('learning', 0.2622)]
```

```
print(type(keywords))
```

```
<class 'list'>
```

```
print(type(keywords[0]))
```

```
<class 'tuple'>
```

```
kw_model.extract_keywords(text,keyphrase_ngram_range=(1,3),stop_words="english")
```

```
[('tomographic imaging deep', 0.6704),
 ('imaging deep learning', 0.6543),
 ('learning medical imaging', 0.6041),
 ('imaging deep', 0.5919),
 ('medical imaging', 0.5281)]
```

```
print(keywords)
```

```
[('imaging', 0.4528), ('tomographic', 0.4488), ('reconstruction', 0.3623), ('deep', 0.3003), ('learning', 0.2622)]
```

```
def search_and_summarize(query,k=5):
    query_embedding=model.encode([query])
    faiss.normalize_L2(query_embedding)
    D,I=index.search(query_embedding,k)
    for score,idx in zip(D[0],I[0]):
        print("Similarity score",score)
        print("Title",df.iloc[idx]["title"])
        print("Abstract",df.iloc[idx]["abstract"][:500])
        print()
        summary=summarizer(df.iloc[idx]["abstract"],max_length=120,min_length=40)
        print(summary)
```

```
print(summary[0]["summary_text"])
print()
keywords=kw_model.extract_keywords(text,keyphrase_ngram_range=(1,3),stop_words="english")
print(keywords)
for k,s in keywords:
    print(k)
```

Start coding or [generate](#) with AI.